

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

UNIT 6

Lưu trữ dữ liệu

ONE LOVE. ONE FUTURE.

Mục đích

- Tìm hiểu về thư viện Flow, Room và DataStore
- Lưu trữ dữ liệu quan hệ
- Lưu trữ dữ liệu đơn giản dạng key-value

Nội dung

- Sử dụng thư viện Room để lưu dữ liệu cấu trúc
- Lưu trữ và truy nhập dữ liệu key-value với DataStore

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 1

GIỚI THIỆU VỀ SQL

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 2

SỬ DỤNG THƯ VIỆN ROOM ĐỂ LƯU TRỮ DỮ LIỆU

- Kotlin Flow
- Thao tác dữ liệu với Room

Kotlin Flow

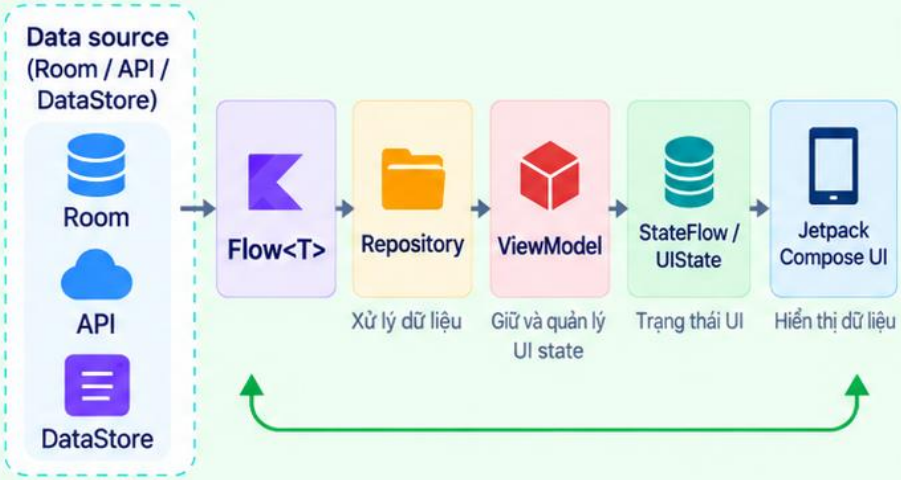
Khái niệm



1. Flow là luồng dữ liệu bất đồng bộ phát ra theo thời gian
2. Có thể phát 0, 1 hoặc nhiều giá trị
3. Tích hợp tốt với Kotlin Coroutines

Luồng dữ liệu với Flow

Từ dữ liệu đến trải nghiệm liên mạch



Flow giúp dữ liệu được quan sát liên tục và cập nhật tự động lên UI.

Vai trò

- Quan sát dữ liệu thay đổi theo thời gian
- Kết nối dữ liệu từ Room, DataStore, API
- Giảm callback, code rõ ràng hơn
- Phù hợp với mô hình reactive UI trong Compose

Khi nào nên dùng?

Flow là lựa chọn phù hợp khi bạn làm việc với các nguồn dữ liệu bất đồng bộ và cần cập nhật UI liên tục.

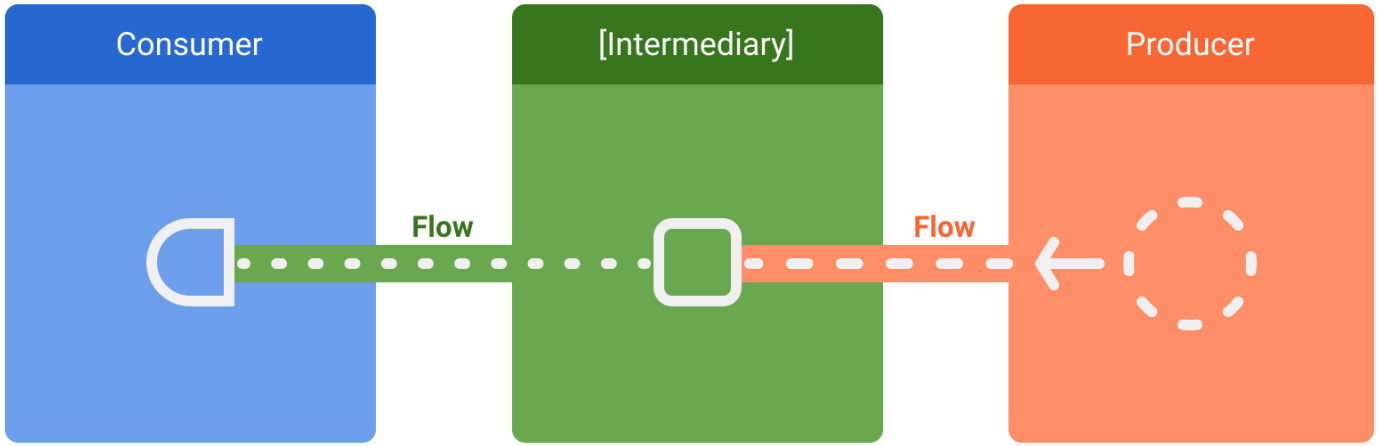
Room | DataStore | Network stream | UI state updates

Flow là gì?

- Flow là một API của **Kotlin Coroutines** dùng để biểu diễn **luồng dữ liệu bất đồng bộ theo thời gian**.
- Một Flow<T> có thể phát ra **nhiều giá trị kiểu T**, thay vì chỉ trả về một giá trị.
- Dữ liệu được phát bằng emit().
- Dữ liệu được nhận bằng collect().
- Flow mặc định là **cold stream**:
  - Code bên trong flow {} chỉ chạy khi có collect().
  - Mỗi collector có thể kích hoạt một luồng thực thi riêng.

Flow hoạt động như thế nào?

- Một **producer** đưa dữ liệu vào luồng, có thể thực hiện bất đồng bộ sử dụng coroutines.
- **(Tùy chọn) Intermediaries** có thể thay đổi mối giá trị được phát vào luồng.
- Một **consumer** thu nhận dữ liệu từ luồng.



Ví dụ Flow

```
fun numberFlow(): Flow<Int> = flow {
    for (number in 1..10) {
        delay(2000)
        emit(number) // Producer phát dữ liệu
    }
}

fun main() = runBlocking {
    numberFlow().collect { number ->
        println("Nhận được: $number") // Collector nhận dữ liệu
    }
}
```

Các toán tử thường được sử dụng với Flow

map() – Biến đổi mỗi giá trị

filter() – Lọc giá trị thỏa mãn điều kiện

distinctUntilChanged() – Bỏ qua các giá trị trùng lặp

combine() – Kết hợp 2 flow

zip() - Ghép cặp 2 flow

flatMapLatest() – Tạo flow mới với mỗi giá trị phát ra, bỏ qua flow cũ.

etc.

Ví dụ

```
flowOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10).map { number ->
    number * number
}.collect { value ->
    print("$value ")
}
```

```
flowOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10).filter { number ->
    number % 2 == 1
}.collect { value ->
    print("$value ")
}
```

```
flowOf(1, 1, 2, 2, 2, 3, 3, 4, 5).distinctUntilChanged().collect { value ->
    print("$value ")
}
```

Ví dụ - combine()

```
val numberFlow = flow {
    for (number in 1..10) {
        delay(1000)
        emit(number) // Producer phát dữ liệu
    }
}
val alphabetFlow = flow {
    for (character in 'A'..'M') {
        delay(1500)
        emit(character) // Producer phát dữ liệu
    }
}
numberFlow.combine(alphabetFlow) { a, b -> "$a$b" }.collect { println("$it ") }
```

Kết quả:

1A 2A 3A 3B 4B 4C 5C 6C 6D 7D 7E 8E 9E 9F 10F 10G 10H 10I 10J 10K 10L 10M

Ví dụ - zip()

```
val numberFlow = flow {
    for (number in 1..10) {
        delay(1000)
        emit(number) // Producer phát dữ liệu
    }
}
val alphabetFlow = flow {
    for (character in 'A'..'M') {
        delay(1500)
        emit(character) // Producer phát dữ liệu
    }
}
numberFlow.zip(alphabetFlow) { a, b -> "$a$b" }.collect { println("$it ")}
```

Kết quả

1A 2B 3C 4D 5E 6F 7G 8H 9I 10J

Ví dụ - flatMapLatest()

```
fun search(keyword: String): Flow<String> = flow {
    repeat(3) {
        delay(1000)
        emit("Result for $keyword - $it")
    }
}

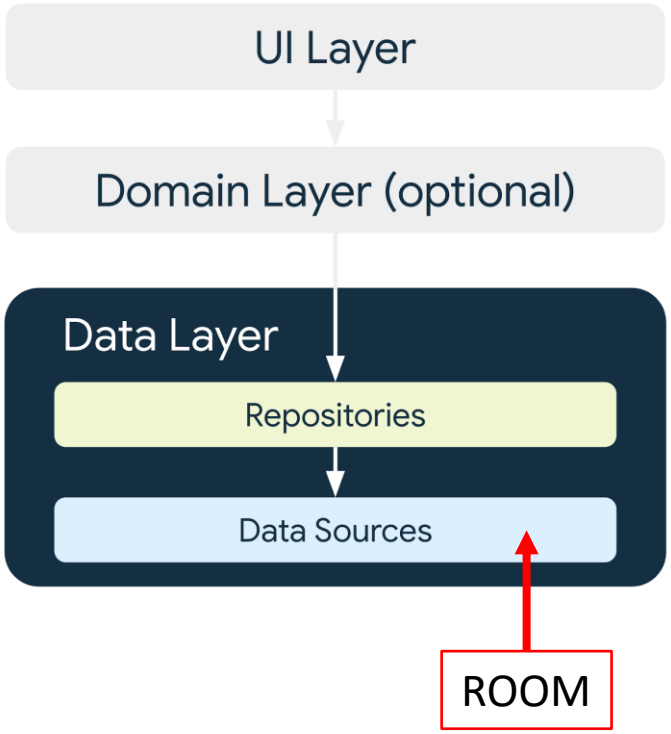
fun main() = runBlocking {
    val queryFlow = flow {
        emit("A")
        delay(1500)
        emit("B")
    }

    queryFlow.flatMapLatest { keyword -> search(keyword) }.collect { println(it) }
}
```

Kết quả:
Result for A - 0
Result for B - 0
Result for B - 1
Result for B - 2

Giới thiệu về thư viện Room

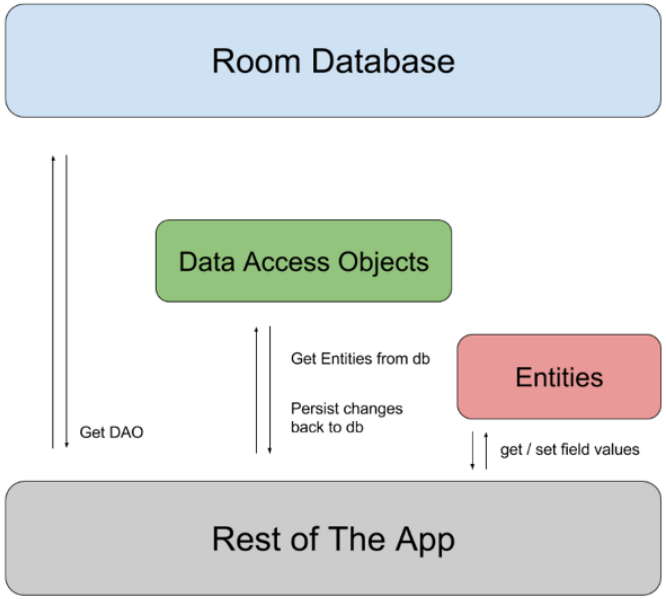
- Là thư viện lưu trữ dữ liệu thuộc Android Jetpack.
- Là lớp trừu tượng dựa trên thư viện SQLite, sử dụng SQL để thực hiện thao tác CSDL.
- Đơn giản hóa quá trình cài đặt, cấu hình và tương tác trong ứng dụng.
- Cung cấp cơ chế kiểm tra cú pháp SQL khi biên dịch.
- Trong kiến trúc ứng dụng, Room là một nguồn dữ liệu.



Các thành phần chính của Room

Bao gồm 3 thành phần chính:

- **Các thực thể (Entites)** – Biểu diễn các bảng trong CSDL.
- **Đối tượng truy nhập (Data Access Object – DAO)** – Cung cấp các phương thức để truy xuất, cập nhật, tạo mới và xóa dữ liệu.
- **Lớp CSDL (Database class)** – cung cấp các đối tượng truy nhập gắn với CSDL.



Khai báo các phụ thuộc của Room

Trong file **build.gradle.kts** (project)

```
plugins {
    id("com.google.devtools.ksp") version "2.3.4" apply false
}
```

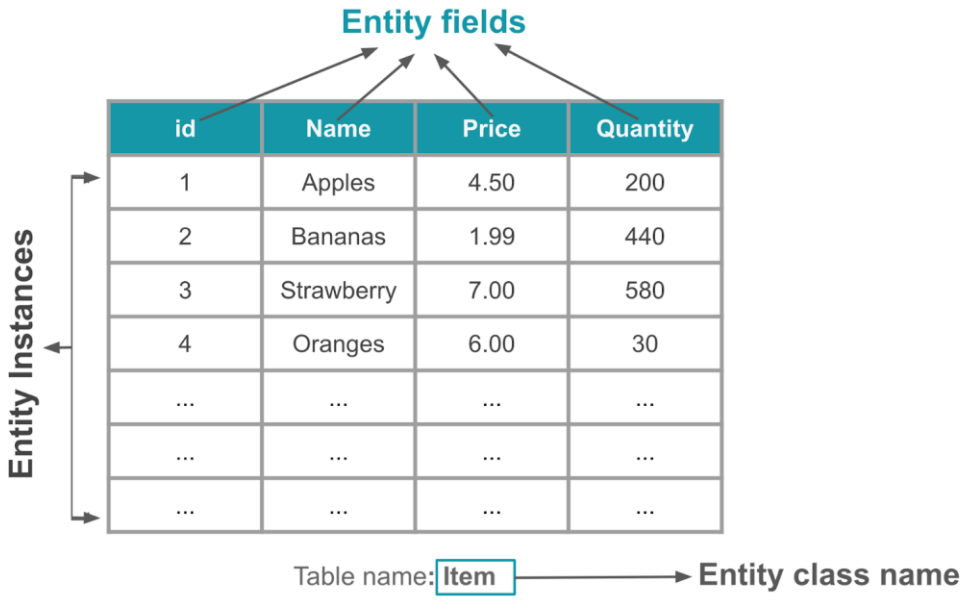
Trong file **build.gradle.kts** (app)

```
plugins {
    id("com.google.devtools.ksp")
}

dependencies {
    implementation("androidx.room:room-runtime:2.8.5")
    ksp("androidx.room:room-compiler:2.8.5")
    implementation("androidx.room:room-ktx:2.8.5")
}
```

Tạo thực thể dữ liệu

- Một lớp Entity định nghĩa một bảng dữ liệu
- Sử dụng data class để định nghĩa lớp
- Một đối tượng thuộc lớp biểu diễn một dòng dữ liệu
- **@Entity** đánh dấu 1 lớp là một Entity
- Mỗi thuộc tính biểu diễn một cột trong bảng
- Sử dụng **@PrimaryKey** để định nghĩa khóa chính



Ví dụ

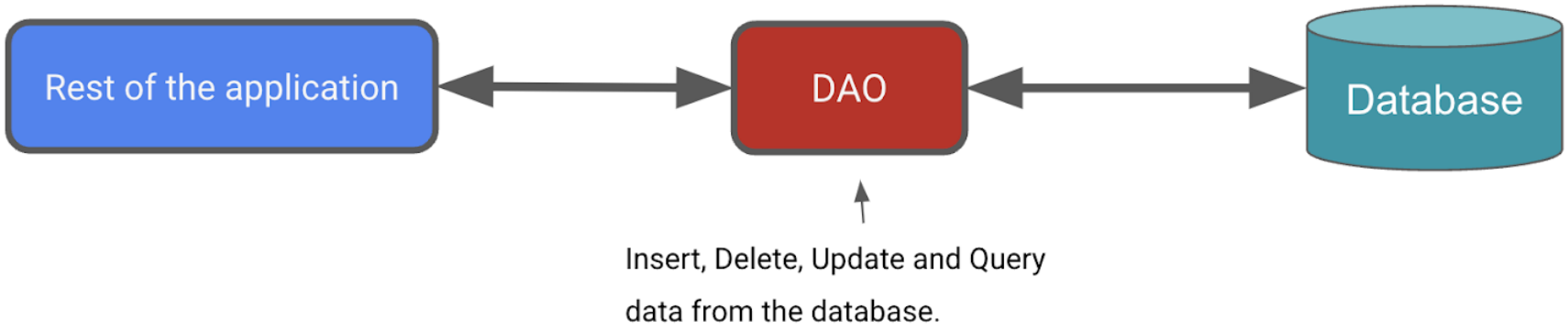
```
@Entity(tableName = "items")
data class Item(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val name: String,
    val price: Double,
    val quantity: Int
)
```

Chú ý:

- **tableName** cho phép đặt tên bảng khác tên lớp
- **autoGenerate** thiết lập việc tự sinh giá trị khóa chính (cần thiết lập giá trị mặc định là 0)
- Có thể sử dụng **@ColumnInfo** để thiết lập tên cột khác tên thuộc tính (trường)

Tạo đối tượng DAO

- Mẫu thiết kế được sử dụng để tách biệt lớp lưu trữ dữ liệu với phần còn lại của ứng dụng.
- Định nghĩa các thao tác truy nhập CSDL: truy vấn, thêm, xóa và cập nhật.
- Định nghĩa như một interface hoặc lớp trừu tượng.
- Room triển khai và kiểm tra lỗi trong quá trình dịch.



Tạo đối tượng DAO

- Sử dụng **@Dao** để đánh dấu lớp là một **DAO**.
- Sử dụng **@Insert**, **@Delete**, **@Update** định nghĩa các phương thức thêm, xóa, sửa mà không cần viết câu truy vấn.
- Sử dụng **@Query** để thực hiện truy vấn hoặc các câu lệnh phức tạp.

Ví dụ

```
@Dao
interface ItemDao {
    @Insert
    suspend fun insert(item: Item): Long
    @Update
    suspend fun update(item: Item): Int
    @Delete
    suspend fun delete(item: Item): Int

    @Query("select * from items order by name asc")
    fun getItems(): Flow<List<Item>>
}
```

Ví dụ

```
@Query("select * from items where id = :id")
fun getItem(id: Int): Flow<Item>

@Query("select * from items where name like '%' || :keyword || '%'")
fun searchItems(keyword: String)
```

- Đây là cách truyền tham số cho phương thức DAO
- Ký tự || dùng để ghép xâu trong SQL
- Ký tự % để thay thế 1 hoặc nhiều ký tự
- Ký tự _ để thay thế 1 ký tự

Tạo lớp cơ sở dữ liệu

- Định nghĩa danh sách các thực thể và DAO.
- Cung cấp DAO để ứng dụng truy nhập CSDL.
- Được định nghĩa theo thiết kế singleton:
  - Khai báo như một lớp trừu tượng, đánh dấu bởi @Database
  - Định nghĩa phương thức trừu tượng để trả về đối tượng thuộc lớp DAO
  - Do chỉ cần một đối tượng thuộc lớp CSDL dùng chung cho ứng dụng => Sử dụng kỹ thuật singleton

Ví dụ

```
@Database(entities = [Item::class], version = 1, exportSchema = false)
abstract class InventoryDatabase: RoomDatabase() {
    abstract fun itemDao(): ItemDao
    companion object {
        @Volatile
        private var Instance: InventoryDatabase? = null

        fun getDatabase(context: Context): InventoryDatabase {
            return Instance ?: synchronized(this) {
                Room.databaseBuilder(context, InventoryDatabase::class.java, "item_db")
                    .build()
                    .also { Instance = it }
            }
        }
    }
}
```

Giải thích

```
@Database(entities = [Item::class], version = 1, exportSchema = false)
```

entities – liệt kê danh sách các thực thể

version – khi thay đổi cấu trúc bảng, cần tăng số hiệu phiên bản

exportSchema – thiết lập lưu trữ cấu trúc bảng

```
@Volatile
private var Instance: InventoryDatabase? = null
```

Biến khai báo bằng **@Volatile** không được cached, mà truy nhập trực tiếp từ bộ nhớ chính => đảm bảo giá trị luôn cập nhật từ tất cả các luồng.

```
return Instance ?: synchronized(this)
```

Đảm bảo tại một thời điểm chỉ có một luồng thực hiện đoạn mã khởi tạo.

Khai báo trong repository

```
interface ItemsRepository {
    suspend fun insertItem(item: Item?): Long
    suspend fun updateItem(item: Item?): Int
    suspend fun deleteItem(item: Item?): Int
    fun getAllItemsStream(): Flow<List<Item>>
}


class OfflineItemsRepository(private val itemDao: ItemDao): ItemsRepository {
    override suspend fun insertItem(item: Item?): Long =
        item?.let { itemDao.insert(item) } ?: -1
    override suspend fun deleteItem(item: Item?): Int =
        item?.let { itemDao.delete(item) } ?: -1
    override suspend fun updateItem(item: Item?): Int =
        item?.let { itemDao.update(item) } ?: -1
    override fun getAllItemsStream(): Flow<List<Item>> =
        itemDao.getItems()
}
```

Khai báo lớp Application để khởi tạo DB

```
class InventoryApplication: Application() {
    lateinit var itemDao: ItemDao
    val repository: ItemsRepository by lazy {
        OfflineItemsRepository(itemDao)
    }

    override fun onCreate() {
        super.onCreate()

        itemDao = InventoryDatabase.getDatabase(this).itemDao()
    }
}
```

Ánh xạ từ Flow sang StateFlow trong ViewModel

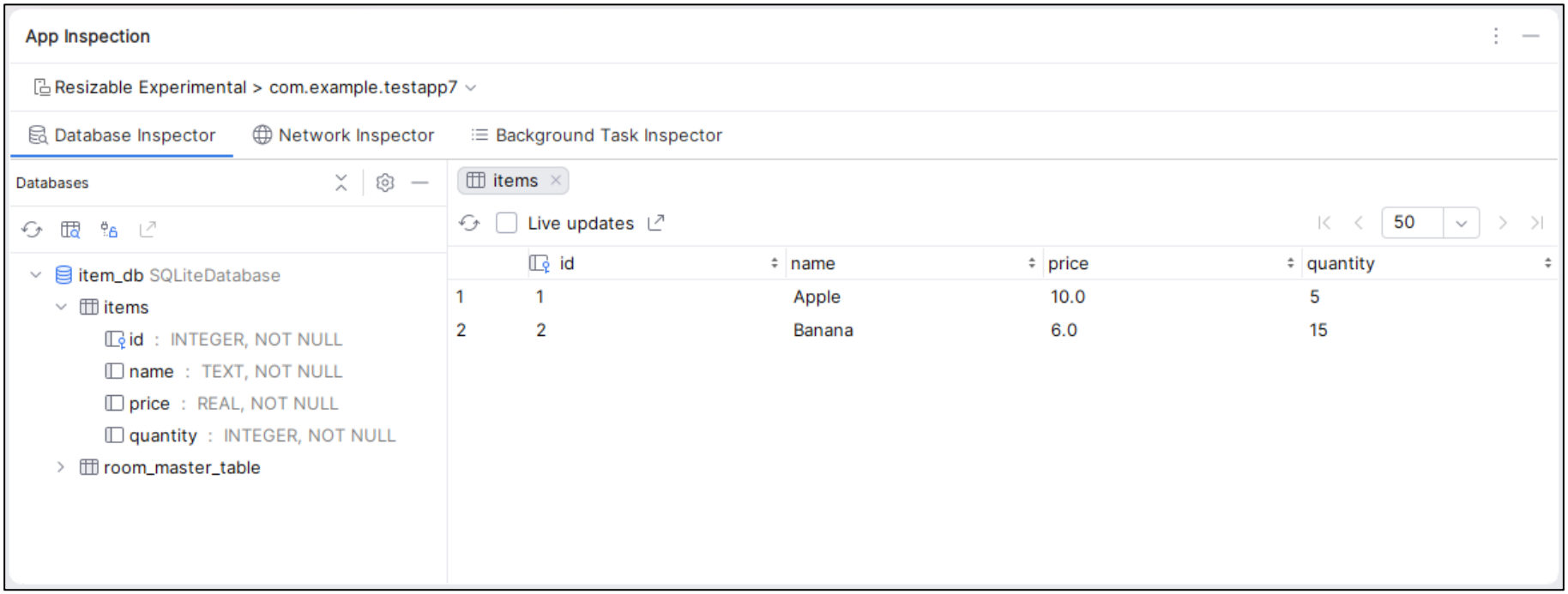
```
class ItemViewModel(private val repository: ItemsRepository): ViewModel() {
    val items: StateFlow<List<Item>> = repository.getAllItemsStream().stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),
        initialValue = listOf()
    )
    ...
    fun addItem() {  viewModelScope.launch {
        repository.insertItem(_uiState.value.item)
    }}
    fun updateItem() { viewModelScope.launch {
        repository.updateItem(_uiState.value.item)
    }}
    fun deleteItem() { viewModelScope.launch {
        repository.deleteItem(_uiState.value.item)
    }}
    ...
}
```

Tạo factory khởi tạo ViewModel

```
class ItemViewModel(private val repository: ItemsRepository): ViewModel() {
    ...
    companion object {
        val Factory: ViewModelProvider.Factory = viewModelFactory {
            initializer {
                val application = (this[APPLICATION_KEY] as InventoryApplication)
                ItemViewModel(application.repository)
            }
        }
    }
}
```

Xem nội dung CSDL với Database Inspector

- Áp dụng với API 27 trở lên
- Cho phép quan sát CSDL trong quá trình chạy
- Mở App Inspection, chọn Database Inspector



HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 3

LƯU TRỮ VÀ TRUY NHẬP DỮ LIỆU KEY-VALUE SỬ DỤNG DATASTORE

Thư viện DataStore

- DataStore là giải pháp lưu trữ dữ liệu nhỏ theo dạng **key-value** hoặc **typed objects**.
- Được thiết kế để thay thế dần **SharedPreferences**.
- Hoạt động theo cơ chế **asynchronous**.
- Sử dụng **Kotlin Coroutines** và **Flow** để đọc/ghi dữ liệu.
- Phù hợp với:
  - User preferences
  - App settings
  - Theme / language
  - Login state hoặc các cấu hình nhỏ

So sánh DataStore và SharedPreferences



DataStore

Modern • Safe • Asynchronous

API	Asynchronous
UI Thread	Không block UI thread
Data Stream	Hỗ trợ Flow
Reliability	Transactional, an toàn hơn
Data Model	Preferences / Proto
Recommendation	Khuyến nghị cho app mới



SharedPreferences

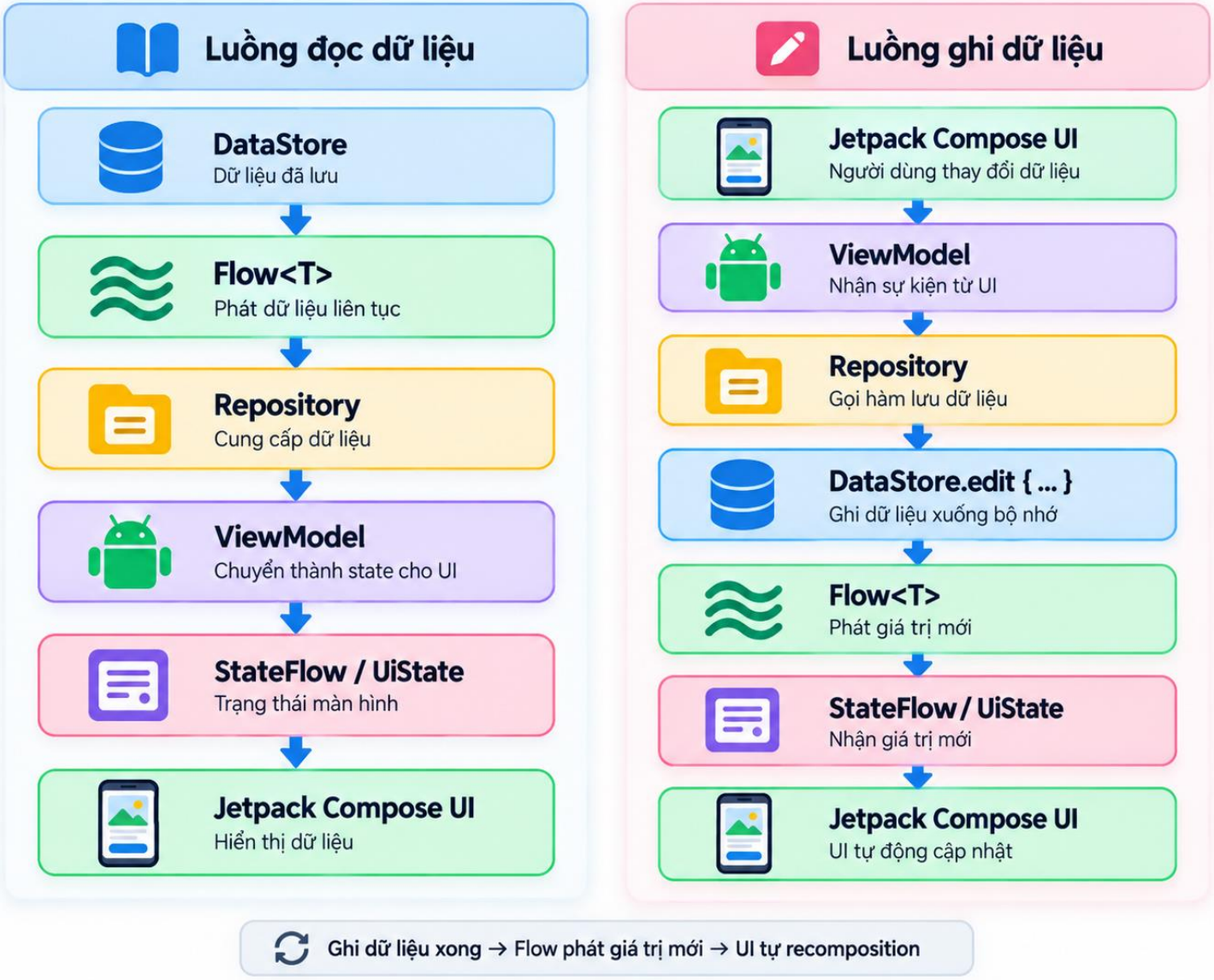
Traditional • Simple • Synchronous

API	Synchronous
UI Thread	Có thể block UI thread
Data Stream	Không hỗ trợ Flow
Reliability	Dễ lỗi khi đọc/ghi
Data Model	Key-Value
Recommendation	Giải pháp cũ / legacy



DataStore là lựa chọn hiện đại hơn cho việc lưu trữ settings trong Android.

Luồng dữ liệu khi sử dụng DataStore



Khởi tạo phụ thuộc trong build.gradle.kts

```
implementation("androidx.datastore:datastore-preferences:1.2.1")
```

Khai báo repository chứa DataStore

```
class PreferencesRepository(private val dataStore: DataStore<Preferences>) {
    // Define the keys for your preferences
    private companion object {
        val IS_LINEAR_LAYOUT = booleanPreferencesKey("is_linear_layout")
    }

    // Save the preference to DataStore
    suspend fun saveLayoutPreference(isLinearLayout: Boolean) {
        dataStore.edit {
            preferences -> preferences[IS_LINEAR_LAYOUT] = isLinearLayout
        }
    }

    // Load the preference from DataStore
    val isLinearLayout: Flow<Boolean> =
        dataStore.data.map { preferences ->
            preferences[IS_LINEAR_LAYOUT] ?: false
        }
}
```

Khởi tạo repository trong lớp Application

```
private const val LAYOUT_PREFERENCE_NAME = "layout_preferences"
private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(
    name = LAYOUT_PREFERENCE_NAME
)

class StudentApplication: Application() {
    lateinit var preferencesRepository: PreferencesRepository
    override fun onCreate() {
        super.onCreate()

        preferencesRepository = PreferencesRepository(dataStore)
    }
}
```

Khai báo trong ViewModel

```
class StudentViewModel(private val preferencesRepository:
PreferencesRepository): ViewModel() {
    companion object {
        val Factory: ViewModelProvider.Factory = viewModelFactory {
            initializer {
                val application = (this[APPLICATION_KEY] as StudentApplication)
                StudentViewModel(application.preferencesRepository)
            }
        }
    }
    ...
}
```

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-flight-search

HUST

hust.edu.vn fb.com/dhbkhn

Q&A